

Core Math Behind `vicon_goto_mppi_controller.py`

June 22, 2026

1 Purpose

This note explains the core control math implemented in `scripts/vicon_goto_mppi_controller.py`. The goal of that script is to drive a differential-drive JetBot from its current Vicon pose to a planar target (x_d, y_d) by repeatedly solving a short finite-horizon control problem with Model Predictive Path Integral (MPPI) control.

The main code blocks discussed here are:

- heuristic command seed: lines 236–264,
- rollout dynamics model: lines 266–283,
- rollout cost: lines 285–326,
- heuristic rollout construction: lines 328–343,
- MPPI update: lines 345–419,
- outer control loop: lines 421–489.

2 State, Target, and Heading

In the outer loop (lines 443–456), the script reads the latest Vicon pose and computes:

$$\Delta x = x_d - x, \quad \Delta y = y_d - y, \quad r = \sqrt{(\Delta x)^2 + (\Delta y)^2}.$$

The target heading is

$$\theta_d = \text{atan2}(\Delta y, \Delta x).$$

The robot heading is obtained from the Vicon orientation by rotating the body Y axis into the world frame, projecting onto the floor plane, and applying a user-entered angle correction. In compact form,

$$\theta = \text{wrapToPi}(\text{atan2}(f_y, f_x) + \delta),$$

where (f_x, f_y) is the horizontal forward vector and δ is the angle correction. The heading error is then

$$e_\theta = \text{wrapToPi}(\theta_d - \theta).$$

If $r \leq \varepsilon$, the controller stops immediately (lines 458–469).

3 Heuristic Command Used as the MPPI Seed

Before sampling random control perturbations, the script builds a simple goal-seeking differential-drive command in `_heuristic_command()` (lines 236–264). This heuristic is not the final controller by itself; it is the warm-start policy that biases MPPI toward sensible commands.

3.1 Distance-Based Forward Speed

Let:

- $v_{\max} = \text{self}._{\text{speed}}$,
- $v_{\min} = \text{self}._{\text{min_forward_speed}}$,
- $r_s = \text{self}._{\text{slow_radius}}$.

Then the base forward command is

$$v_{\text{base}} = \begin{cases} v_{\max}, & r \geq r_s, \\ v_{\min} + (v_{\max} - v_{\min}) \frac{r}{r_s}, & r < r_s. \end{cases}$$

This matches lines 248–253.

3.2 Heading Alignment and Steering

The same block computes

$$a = \max(0, \cos e_\theta), \quad v = v_{\text{base}} a^2,$$

so the robot reduces forward motion when it is poorly aligned with the target (lines 255–258).

Steering is proportional to heading error and then clamped:

$$\omega = \text{clamp}(k_\theta e_\theta, -u_{\text{turn}}, u_{\text{turn}}),$$

where $k_\theta = \text{self}._{\text{heading_gain}}$ and $u_{\text{turn}} = \text{self}._{\text{turn_speed}}$ (lines 259–263).

The heuristic left and right wheel commands are

$$u_L = \text{clamp}(v - \omega), \quad u_R = \text{clamp}(v + \omega),$$

as returned on line 264. Here the one-argument `clamp()` means clamp to the motor command range, effectively $[-1, 1]$.

4 Internal Rollout Dynamics Model

The MPPI controller needs a fast predictive model to evaluate candidate command sequences. The model in `_simulate_step()` (lines 266–283) uses a simple differential-drive abstraction.

For wheel commands (u_L, u_R) :

$$u_f = \frac{u_L + u_R}{2}, \quad u_\omega = \frac{u_R - u_L}{2}.$$

These are interpreted as forward and turn commands. They are converted into physical rates by calibration constants `self._model_linear_speed` and `self._model_angular_speed`:

$$v = c_v u_f, \quad \omega = c_\omega u_\omega.$$

Over one controller period $\Delta t = \text{self}.\text{period}$, the predicted state update is

$$\begin{aligned}x_{k+1} &= x_k + v \cos(\theta_k) \Delta t, \\y_{k+1} &= y_k + v \sin(\theta_k) \Delta t, \\\theta_{k+1} &= \text{wrapToPi}(\theta_k + \omega \Delta t).\end{aligned}$$

This is exactly what lines 274–282 implement.

5 Rollout Cost Function

The cost of a candidate control sequence is computed in `_sequence_cost()` (lines 285–326). Suppose a rollout consists of

$$\mathbf{u}_{0:H-1} = \{(u_{L,0}, u_{R,0}), \dots, (u_{L,H-1}, u_{R,H-1})\}.$$

At each rollout step k , the script simulates the next state and evaluates:

- distance-to-goal penalty,
- heading-error penalty,
- control-effort penalty,
- control-slew penalty relative to the previous command,
- reverse-motion penalty.

5.1 Stage Cost

Let

$$r_k = \sqrt{(x_d - x_k)^2 + (y_d - y_k)^2}, \quad e_{\theta,k} = \text{wrapToPi}(\text{atan2}(y_d - y_k, x_d - x_k) - \theta_k).$$

The code scales distance by

$$\bar{r}_k = \frac{r_k}{r_{\text{scale}}}, \quad r_{\text{scale}} = \text{self}.\text{distance_scale},$$

with `self.distance_scale` initialized from the slow radius.

Let the previous command be $(u_{L,k-1}, u_{R,k-1})$. Then the stage cost used by lines 308–315 can be written as

$$\begin{aligned}\ell_k &= w_r \bar{r}_k^2 + w_\theta e_{\theta,k}^2 + w_u (u_{L,k}^2 + u_{R,k}^2) \\ &\quad + w_{\Delta u} ((u_{L,k} - u_{L,k-1})^2 + (u_{R,k} - u_{R,k-1})^2) + w_{\text{rev}} \max(-u_{f,k}, 0)^2,\end{aligned}$$

where

$$u_{f,k} = \frac{u_{L,k} + u_{R,k}}{2}.$$

The weights above correspond to:

- `self.distance_cost_weight`,
- `self.heading_cost_weight`,
- `self.control_effort_weight`,
- `self.control_slew_weight`,
- `self.reverse_cost_weight`.

5.2 Terminal Cost

After the final simulated step, lines 319–325 add a terminal penalty:

$$\phi = w_r^{(T)} \bar{r}_H^2 + w_\theta^{(T)} e_{\theta,H}^2,$$

using `self._terminal_distance_cost_weight` and `self._terminal_heading_cost_weight`.

The full rollout objective is therefore

$$J(\mathbf{u}_{0:H-1}) = \sum_{k=0}^{H-1} \ell_k + \phi.$$

6 How the Heuristic Becomes a Nominal Sequence

Lines 328–343 build a horizon-length heuristic sequence by repeatedly applying the heuristic command and rolling the internal model forward:

$$\mathbf{u}_{0:H-1}^{\text{heur}} = \{\pi_{\text{heur}}(x_0, y_0, \theta_0), \pi_{\text{heur}}(x_1, y_1, \theta_1), \dots\}.$$

Then lines 354–361 combine:

- the previous cycle’s shifted nominal sequence, and
- the fresh heuristic sequence.

If $\alpha = \text{self._heuristic_blend}$, the nominal sequence at step k is:

$$\mathbf{u}_k^{\text{nom}} = (1 - \alpha) \mathbf{u}_k^{\text{shift}} + \alpha \mathbf{u}_k^{\text{heur}}.$$

This warm start is important: it makes the sampled rollouts less random and gives MPPI a plausible local baseline.

7 MPPI Sampling and Softmin Update

The core MPPI logic lives in `_mppi_command()` (lines 345–419).

7.1 Sampling Candidate Sequences

For each sample index $i \in \{1, \dots, N\}$ and horizon step k , the script draws Gaussian noise

$$\epsilon_k^i = \begin{bmatrix} \epsilon_{L,k}^i \\ \epsilon_{R,k}^i \end{bmatrix}, \quad \epsilon_{L,k}^i, \epsilon_{R,k}^i \sim \mathcal{N}(0, \sigma^2),$$

where $\sigma = \text{self._noise_std}$ (lines 370–379).

The sampled command is

$$\mathbf{u}_k^i = \text{clamp}(\mathbf{u}_k^{\text{nom}} + \epsilon_k^i).$$

Each full candidate sequence is then scored by the rollout cost

$$J_i = J(\mathbf{u}_{0:H-1}^i),$$

as in lines 381–389.

7.2 Exponential Weighting

After evaluating all candidates, the controller computes the minimum cost

$$J_{\min} = \min_i J_i$$

and then assigns each rollout a softmax weight (lines 393–395):

$$w_i = \exp\left(-\frac{J_i - J_{\min}}{\lambda}\right),$$

where $\lambda = \text{self.temperature}$.

Subtracting $J_{\min}$ does not change the relative weighting, but it improves numerical stability by keeping exponents closer to zero.

7.3 Weighted Noise Update

For each horizon step k , the script computes the weighted mean noise:

$$\bar{\epsilon}_k = \frac{\sum_{i=1}^N w_i \epsilon_k^i}{\sum_{i=1}^N w_i}.$$

The nominal sequence is updated by

$$\mathbf{u}_k^{\text{new}} = \text{clamp}(\mathbf{u}_k^{\text{nom}} + \bar{\epsilon}_k),$$

which is what lines 400–416 implement.

Finally, line 419 returns only the first control:

$$\mathbf{u}_{\text{apply}} = \mathbf{u}_0^{\text{new}}.$$

This is the standard receding-horizon idea: optimize a short sequence, apply the first command, shift the horizon, and repeat on the next cycle.

8 Outer Controller Loop

The outer loop in `run()` (lines 421–489) connects the math to the live robot:

1. get the active target and current tuning (lines 423–424),
2. wait if no target is active (lines 426–428),
3. read the latest Vicon pose and verify visibility (lines 430–441),
4. compute current planar state and heading error (lines 443–456),
5. stop if the robot is already within ε (lines 458–469),
6. read the currently commanded wheel speeds (line 472),
7. call `mppi_command(...)` to get the next wheel command (lines 473–480),
8. send the command to the JetBot and update status (lines 482–488).

The current wheel command is passed into the cost function so the control-slew term can discourage abrupt changes.

9 Compact Summary

The controller can be summarized as:

measure current state $\rightarrow (x, y, \theta)$,
build heuristic seed $\rightarrow \mathbf{u}_{0:H-1}^{\text{heur}}$,
blend with shifted previous plan $\rightarrow \mathbf{u}_{0:H-1}^{\text{nom}}$,
sample noisy sequences $\rightarrow \mathbf{u}_{0:H-1}^i$,
score each sequence $\rightarrow J_i$,
compute softmax weights $\rightarrow w_i$,
update the nominal plan $\rightarrow \mathbf{u}_{0:H-1}^{\text{new}}$,
apply the first command $\rightarrow (u_L, u_R)$.

So the script is not using an analytic optimal-control solution. Instead, it uses a simple dynamics model plus many short random control perturbations, then keeps the perturbation directions that tend to reduce the finite-horizon cost.

10 Parameter Anchors in the File

The MPPI-related tunable parameters are declared near lines 526–550:

- speed and turning limits: lines 526–533,
- horizon and sample count: lines 537–540,
- model calibration: lines 541–542,
- heuristic blending: line 543,
- distance, heading, effort, slew, reverse, and terminal weights: lines 544–550.

Those parser defaults determine the numerical behavior of the equations above.